

Name: Jabir Hussain , RollNo.: 2447012

Exp19:Write a program to perform Logistic Regression on “Pima Indians Diabetes” and “Iris” dataset

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

print("Shape of X:", X.shape)
print("Shape of y:", y.shape)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Logistic Regression model
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)
```

Predictions

```
y_pred = model.predict(X_test)
```

Accuracy & Report

```
print("\nAccuracy:", accuracy_score(y_test, y_pred))
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred,  
target_names=iris.target_names))
```

Output:

```
➡ Shape of X: (150, 4)  
Shape of y: (150,)  
  
Accuracy: 1.0  
  
Classification Report:  
              precision    recall  f1-score   support  
  
   setosa      1.00      1.00      1.00        19  
  versicolor  1.00      1.00      1.00        13  
   virginica   1.00      1.00      1.00        13  
  
   accuracy                1.00        45  
  macro avg      1.00      1.00      1.00        45  
 weighted avg      1.00      1.00      1.00        45
```

```
from sklearn.datasets import load_diabetes

# Load the data as numpy arrays (default scaled)
A, b = load_diabetes(return_X_y=True, as_frame=False, scaled=True)

print("Shape of A:", A.shape)
print("Shape of b:", b.shape)
print("First 3 samples in A:")
print(A[:3])
print("First 3 target values:")
print(b[:3])
```

Output:

```
▶
↳ Shape of A: (442, 10)
Shape of b: (442,)
First 3 samples in A:
[[ 0.03807591  0.05068012  0.06169621  0.02187239 -0.0442235  -0.03482076
 -0.04340085 -0.00259226  0.01990749 -0.01764613]
 [-0.00188202 -0.04464164 -0.05147406 -0.02632753 -0.00844872 -0.01916334
  0.07441156 -0.03949338 -0.06833155 -0.09220405]
 [ 0.08529891  0.05068012  0.04445121 -0.00567042 -0.04559945 -0.03419447
 -0.03235593 -0.00259226  0.00286131 -0.02593034]]
First 3 target values:
[151.  75. 141.]
```

#for Iris s\data set

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
```

```
clf = LogisticRegression(random_state=0)
```

```
clf.fit(X_train, y_train)
```

output:

```
clf.fit(X_train, y_train)
```



LogisticRegression



LogisticRegression(random_state=0)

```
y_pred = clf.predict(X_test)
```



```
accuracy_score(y_test, y_pred)
```



1.0

10]
/ Os

classification_report(y_test, y_pred)

		precision	recall	f1-score	support	0	1	1.00	1.00	1.00	10	1	1.00	1.0
0	1.00	9	2	1.00	1.00	1.00	11	1.00	accuracy			1.00	30	macro a
vg	1.00	1.00	1.00	30	nweighted avg	1.00	1.00	1.00	30	n'				

```
clf = LogisticRegression(random_state=0)
clf.fit(a_train, b_train)
```

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
  LogisticRegression
LogisticRegression(random_state=0)
```

```
b_pred = clf.predict(a_test)
```

```
accuracy_score(b_test, b_pred)
```

```
0.7402597402597403
```

Exp. Build KNN Classification on the “Iris” dataset

```
import kagglehub

# Download the latest version of the dataset
path = kagglehub.dataset_download("uciml/pima-indians-diabetes-database")

print("Dataset downloaded to:", path)
```

Output:

```
↳ Using Colab cache for faster access to the 'pima-indians-diabetes-database' dataset.
Dataset downloaded to: /kaggle/input/pima-indians-diabetes-database
```

Learning Outcome:

Learn how to model the probability of class membership using the sigmoid function.

Implement Logistic Regression for binary and multiclass classification problems.

Split data into training and testing sets and evaluate performance using:

Classification Report

Interpret model coefficients and understand the relationship between independent variables and the predicted class.

Apply feature scaling and regularization to improve model performance.

Understand how logistic regression differs from linear regression and when to use

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
import os
import kagglehub

# Download the Pima Indians Diabetes dataset from Kaggle
path = kagglehub.dataset_download("uciml/pima-indians-diabetes-database")

# Find the CSV file in the downloaded path
csv_file = [f for f in os.listdir(path) if f.endswith('.csv')][0]
csv_file_path = os.path.join(path, csv_file)

# Load the dataset
df_diabetes = pd.read_csv(csv_file_path)
print("Dataset shape:", df_diabetes.shape)
print(df_diabetes.head())
```

```
# Separate features (X) and target (y)
X_diabetes = df_diabetes.drop('Outcome', axis=1)
y_diabetes = df_diabetes['Outcome']

# Split the dataset into training and testing sets
X_train_d, X_test_d, y_train_d, y_test_d = train_test_split(
    X_diabetes, y_diabetes, test_size=0.2, random_state=42
)

# Create and train the Logistic Regression model
model_diabetes = LogisticRegression(max_iter=1000) # Increased max_iter for convergence
model_diabetes.fit(X_train_d, y_train_d)

# Make predictions
y_pred_d = model_diabetes.predict(X_test_d)

# Evaluate the model
print("\nAccuracy:", accuracy_score(y_test_d, y_pred_d))
print("\nClassification Report:\n", classification_report(y_test_d, y_pred_d))
```

Output:

Using Colab cache for faster access to the 'pima-indians-diabetes-database' dataset.

Dataset shape: (768, 9)

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI \
0	6	148	72	35	0	33.6
1	1	85	66	29	0	26.6
2	8	183	64	0	0	23.3
3	1	89	66	23	94	28.1
4	0	137	40	35	168	43.1

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

Accuracy: 0.7467532467532467

```
Classification Report:

```

	precision	recall	f1-score	support
0	0.81	0.79	0.80	99
1	0.64	0.67	0.65	55
accuracy			0.75	154
macro avg	0.73	0.73	0.73	154
weighted avg	0.75	0.75	0.75	154

```
from sklearn import datasets
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.metrics import accuracy_score
```

```
#step 2: Load the Iris dataset
```

```
iris = datasets.load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
iris = datasets.load_iris()

X = iris.data

y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

#step 4:Normalize feature data

scaler = StandardScaler()

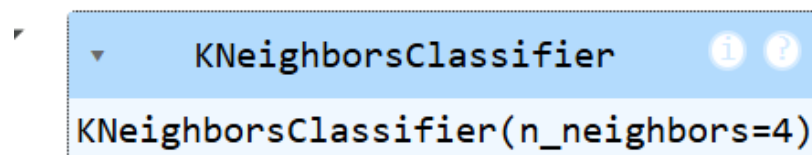
X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)


knn = KNeighborsClassifier(n_neighbors=4)

knn.fit(X_train, y_train)
```

Output:



```
KNeighborsClassifier
```

```
KNeighborsClassifier(n_neighbors=4)
```

```
y_pred = knn.predict(X_test)

acc = accuracy_score(y_test, y_pred)

print(f"kNN Accuracy: {acc}")
```

Output:

 kNN Accuracy: 0.9777777777777777

Learning Outcome:

Understand the concept of instance-based learning, where predictions are made based on the nearest data points.

Split data into training and testing sets and evaluate the model using metrics such as accuracy, precision, recall, and F1-score.

Gain experience in hyperparameter tuning (choosing optimal K) to improve model performance.

Visualize decision boundaries and understand how KNN works with different datasets.

